Join unlimited access to the best of Medium for less than $5/week. Become a member

DuckDB Medallion Architecture: A Complete Local Lakehouse Guide

Tomas Suarez Follow

Local DuckDB medallion architecture in action: bronze, silver, and gold layers processing 1M rows in 4 seconds, creating 5.4GB total across three .duckdb files.

Embedded OLAP. DuckDB is like SQLite for analytics — a single file, vectorized, and ridiculously fast on one machine with zero setup to run.

DuckDB in Context

What's Actually Happening Under the Hood

DuckDB borrows SQLite's "single-file database" philosophy but reimagines it for analytics. Here's what lives inside that `.duckdb` file:

Row groups — Data compressed into columnar chunks (122,880 rows each). Think Parquet blocks, but managed directly by DuckDB.

Block manager — Handles storage allocation like a tiny filesystem. Adds blocks when you insert data, reclaims them when you delete.

Write-Ahead Log (WAL) — Your safety net. Writes go here first, then get flushed to the main file on checkpoint. Crash? Replay the WAL.

Checkpoints — The "save game" moment. Flushes pending writes from WAL into the main database file.

Unlike Magma (Cloudburst's LSM tree key-value engine), DuckDB is columnar and SQL native. Unlike Parquet files alone, it gives you indexes, transactions, and a query optimizer. It's essentially **a self-contained Parquet engine** with SQL on top, or a distributed system pretending to be small.

The Comparison Landscape

vs. Transactional Databases (MySQL, Postgres, Oracle, MongoDB)

These are built for OLTP: thousands of small reads/writes per second, row-by-row updates, strong ACID guarantees. Perfect for web apps, banking systems, and user-facing services.

DuckDB is built for OLAP: scanning millions of rows, aggregations, joins, and analytical queries. Perfect for data science, ETL pipelines, and BI dashboards.

Use Postgres for your API backend. Use DuckDB to analyze what that backend produced.

vs. Cloud Warehouses (Snowflake, BigQuery, Redshift)

These scale out — add more nodes to handle petabytes. Great for enterprise-scale analytics, but you're paying for compute, storage, and network egress. Queries take seconds to minutes, and local dev is awkward (you're always connecting).

Queries take milliseconds, and there's no cloud bill.

Use Snowflake and equivalents cloud-side when you need distributed compute. Use DuckDB when you want instant answers on your machine.

vs. SQLite

SQLite is row-oriented and single-threaded. It's the King of embedded transactional databases — every smartphone has one.

DuckDB is columnar and parallel. It's designed for analytical workloads: reading entire columns, pushing down filters, and vectorized execution. It also speaks Parquet, Arrow, and modern data formats natively.

Use SQLite for application state. Use DuckDB for application analytics.

Why This Setup Works

With just two environment variables (`DATA_ROOT`, `DUCK_DIR`) and two SQL scripts (one to clean prod tables, one to create read-through views), you get:

Instant dev/prod separation — Experiment freely without touching production

Millisecond queries — From six million of rows

Refresh on demand — `./ingest.sh DEV_MODE=1 TEST_ITEMS=10000` and you're synced

You hop between environments like switching Git branches. Prod stays pristine. Dev is your playground. And when you're ready, promoting changes is just running the same SQL in prod.

No drama. No downtime. No surprises.

Architecture Medallion + Local OR

We use a simple medallion layout — Bronze (raw) → Silver (cleaned) → Gold (aggregated) — with two catalogs: a Production warehouse (single DuckDB file) and a Development clone. Optionally, phase a secondary DuckDB file on another SSD as local cloud storage.

The Stack

Infrastructure

DuckDB (embedded OLAP engine) — Parquet data lake (Bronze/Silver/Gold layers) — DuckDual catalogs (file-level branches for data) — Ubuntu 24.04 LTS

Hardware NVMe SSD for data, active quarter — Outside SSD or HDD (cold storage, backups, disaster recovery)

Bronze, silver, and gold layers with a primary DuckDB file and a secondary DuckDB file on another SSD for historical archive storage.

Canonical Paths & Environment

Set these environment variables once and reuse them everywhere:

```
export DATA_ROOT=/home/tsuarez/data/duckdb/warehouse/prod_warehouse.duckdb
export DUCK_DIR=/home/tsuarez/data/duckdb/scripts/prod_warehouse.duckdb
export LOG_DIRS=/home/tsuarez/data/duckdb/logs
alias ingest='./ingest.sh DATA_ROOT=$DATA_ROOT DUCK_DIR=$DUCK_DIR'
```

A few modern ways to Access Your Data (Thanks to Modern Lakehouses)

Connecting to Different DuckDB Environments

You can connect to either DEV or PROD directly from the Linux terminal using the DuckDB CLI.
This is the simplest way to run SQL commands without opening Python or a notebook.

```
# Connect to your DEV environment
duckdb "/data/dev.duckdb" -c "
  SUSE TRADING_DATA;
  SELECT COUNT(*) FROM bronze_ref_flights;
  SELECT COUNT(*) FROM silver_ref_flights;
"

# To work the DuckDB shell:
duckdb -ui
```

💡 Tip:
When inside the DuckDB prompt (the `D shell`), you can execute SQL normally.
Use `help` to list commands, and `.exits` to show available tables.

Views Over Parquet (The Lake)

Create views that read directly from Parquet files. Now files are picked up automatically validate directly:

```
-- DEV
SELECT COUNT(*) AS rows, MIN(source), MAX(source)
FROM read_parquet('/home/tsuarez/data/duckdb/lake/bronze/dt=*/dt_*.parquet');
```

If you prefer to check and preview Parquet contents in Python (e.g., view schema, sample rows, or list partitions), use the reader utility included in this repo:

```
python utils/dt_duck_query.py lake/bronze/parquet_reader.py
```

This will print column metadata and a small data sample, confirming your data lake is ready for warehouse ingestion.

Validate files are there:

```
ls -lh /home/tsuarez/data/duckdb/lake/bronze/dt=*/
du -sh /home/tsuarez/data/duckdb/lake/bronze/
```

Read Through Views (DEV → PROD)

In DEV, create views that read from PROD without copying data. Zero storage in DEV.

duckdb "$DUCK_DIR/dev.duckdb" -c "ATTACH '/data/prod/prod.duckdb' AS prod;

Physical DEV Copies (Optional)

When you need editable local copies of production tables, clone them into your DEV environment.
Re-run this anytime to refresh your DEV data from PROD.

```
duckdb "$DUCK_DIR/dev" -cmd "ATTACH '/data/prod/prod.duckdb' AS prod;"
```

Or execute manually inside DuckDB:

```
-- Attach the warehouse (read-only)
ATTACH '/home/tsuarez/data/duckdb/warehouse/prod/prod_warehouse.duckdb' AS ph (READ_ONLY);

-- Create DEV schema (physical editable copies)
CREATE SCHEMA IF NOT EXISTS bronze;

-- Drop old DEV tables if exists
DROP TABLE IF EXISTS bronze.ref_flights;
DROP TABLE IF EXISTS bronze.ref_airports;
DROP TABLE IF EXISTS bronze.ref_airlines;

CREATE UNIQUE INDEX IF NOT EXISTS ph_ref_flights=
  ON bronze.ref_flights(flight_id);
```

These tables live physically inside your DEV DuckDB file, so you can safely modify or test transformations without touching PROD.

In-Memory vs File-Backed Scratchpads

DuckDB lets you work in two modes:

MODE Contents Persistence **:memory:** Lost when you exit **file-backed** /path/to/scratch.duckdb Changes are saved persistently to disk ✓ Ideal for DEV catalogs and scratchpads. Use `:memory:` for temporary sessions, nothing is written to disk. Great for quick experiments or ad-hoc analytics.

Example:

```
duckdb :memory:
  ATTACH '/home/tsuarez/data/duckdb/warehouse/prod/prod_warehouse.duckdb' AS ph (
  SELECT COUNT(*) FROM ph.bronze_ref_flights;
```

Use `:memory:` for ephemeral analysis, or point to a persistent file when you want your scratchpad DEV catalog to survive reboots.

Verify Catalogs & Schemas

Verify Which Database You're Using

```
duckdb "$DUCK_DIR" -c "SHOW DATABASES;"
```

This shows the file path of each attached database and any attached catalogs.

Then list schemas and tables:

```
SELECT schema_name FROM information_schema.schemata ORDER BY 1;
SELECT table_schema, table_name, table_type FROM
information_schema.tables ORDER BY 1,2;
```

Listing Views Only (DEV/ bronze_views)

```
SELECT table_schema, table_name, "TYPE" AS table_type
FROM information_schema.tables
WHERE table_schema = 'bronze_view'
ORDER BY 2;
```

```
SHOW TABLES;
SELECT table_name, sql FROM duckdb_views WHERE schema_name='bronze';
SELECT table_name, sql FROM duckdb_views WHERE schema_name='silver';
SELECT table_name, sql FROM duckdb_views WHERE schema_name='gold';
```

🚀 Common One-Liners You'll Actually Use a Lot

Open PROD and check counts

```
duckdb "$DUCK_DIR.ph" -c "
  SUSE TRADING_DATA;
  SELECT COUNT(*) FROM bronze_ref_flights;
  SELECT COUNT(*) FROM bronze_ref_flights;
"
```

Open DEV and compare with PROD in one go

```
duckdb "$DUCK_DIR" -c "
  ATTACH '/data/prod/prod.duckdb' AS prod (READ_ONLY);
  SELECT 'dev' AS source, COUNT(*) FROM bronze_ref_flights
  UNION ALL
  SELECT 'prod', COUNT(*) FROM prod.bronze_ref_flights;
"
```

Inspect structure and attached catalogs

```
duckdb "$DUCK_DIR.ph" -c "
  DESCRIBE bronze_ref_flights;
  SELECT database_name FROM information_schemata;
"
```

List all views in bronze schema

```
duckdb "$DUCK_DIR" -c "
  SELECT table_name FROM information_schema.views
  WHERE table_schema='bronze_view';
"
```

Quick Parquet validation

```
duckdb :: "
  SELECT COUNT(*) AS rows FROM read_parquet('$LAKE_ROOT/bronze/dt=*/dt_*/flights.pq
```

Troubleshooting Cheatsheet

- **"Conflicting lock is held"** → Close other DuckDB sessions or notebooks. Also:

```
lsof /home/tsuarez/data/duckdb/warehouse/*.duckdb
```

- **Views show 0 rows** → Check your `DATA_ROOT` path and that the source tables exists in PROD.
- **Parquet views empty** → Confirm files exist in `$LAKE_ROOT` and try a direct:

```
SELECT COUNT(*) FROM read_parquet('/path/to/file.parquet');
```

- Recap: What You Have Now

You've built a **mini data warehouse** with:

1. **A Production DuckDB Warehouse** (`/data/prod/prod_warehouse.duckdb`)
2. **A Development clone** — with either read-through views or physical editable copies
3. **A Parquet data lake** stored by year-quarter **partitioned DuckDB views**

Together, these three layers make up a **local medallion lakehouse** — fast, portable, and cloud-free.

Closing Note

DuckDB proves that modern data engineering doesn't need clusters, credentials, or cloud bills.
A single machine, a few folders, and clear separation between DEV, PROD, and the Lake are enough to build a serious analytics stack.

Simple tools. Real data. Full control.

🔗 Connect with me

Thanks for reading! I share insights on **Azure, multi-cloud architecture, Spark and Python automation** for data & AI systems.

📖 Upwork: Tomas Suarez(Link)
📖 Freelancer: tomassuarez(Link)
📖 GitHub: datawarez(Repository/Tomas Suarez)
📖 Substack: datawarez(Link)
📖 LinkedIn: Tomas Suarez(Link)
📖 Medium: @datawarez(Link)

Duckdb | Linux | Python | Data warehouse | Parquet

Written by Tomas Suarez
0 Followers · 1 Following
Building Azure Data Lakes, ETL pipelines, and multi-cloud analytics systems. Learn Azure, Databricks, Spark, Linux, Python, Terraform, and Kubernetes through real-world projects.

Follow

No responses yet
Write a response
Write a response on Medium?

More from Tomas Suarez

Tomas Suarez 1 hour ago
Zero Trust Policy: Azure Container Registry Access Control

Tomas Suarez 1 hour ago
Changing the right Kernel and Python version

Apps Private Deployment Guide

Jul 29, 2025

Nvidia Driver for Ubuntu

Keeping Ubuntu fast and stable usually comes down to two things: having the right...

Oct 25, 2025 1

Tomas Suarez

The Ultimate Linux Gaming Setup & Partition Guide (NVIDIA, Vulka...

I switched from Windows to Linux and built the ultimate setup for smooth gaming, stabl...

Sep 26, 2025 1

In Azure Made Easy by Tomas Suarez

Using Azure Files with AKS and Managed Identity (SMB)

Using Azure Files with AKS and Managed Identity (SMB)

Feb 9

See all from Tomas Suarez

Recommended from Medium

In AWS in Plain English by Khushbu Shah

Data Engineering Design Patterns You Must Learn in 2026

These are the 8 data engineering design patterns every modern data stack is built on...

Jan 5 1.2K 26

In Activated Thinker by Shane Collins

Why the Smartest People in Tech Are Quietly Panicking Right Now

The water is rising fast, and your free version of ChatGPT is hiding the terrifying,...

Feb 12 13K 546

Agent Native

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code...

Jan 20 1.6K 45

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big...

Something strange happened in early 2026. Apple stores started running low on Mac...

Feb 15 4.8K 81

Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

Jan 18 3.8K 88

Phil | Rentier Digital Automation

I Stopped Vibe Coding and Started "Prompt Contracts" — Claude...

Last Tuesday at 2 AM, I deleted 2,400 lines of code that Claude Code had just generated f...

Feb 10 2.7K 69

See more recommendations